AgentPProf: Operation-Stack Profiling for AI Agent Traces

Abstract

AI-agent executions are no longer well described by one prompt, one session, or one trace tree. A single run can contain prompts, tool calls, GUI actions, browser actions, process events, plans, subagents, safety labels, failure labels, and human subtask boundaries. Existing observability systems usually make one boundary primary, such as a session, span, or prompt tree. Classic profilers make another boundary primary, such as a function call stack. This paper argues that agent profiling needs a smaller abstraction: an `operation` is a weighted record with fields, and an `operation stack` is a recursive projection over those fields chosen at query time. Aggregation is not a separate object or a fixed trace tree; it is the result of choosing a stack shape over the same operations. This is not merely a claim about query-time aggregation, which classic profilers and trace processors also support in different forms. The scoped novelty is the combination of an agent-operation record model, recursive multi-field operation-stack projections, and a hidden-label cross-dataset localization benchmark.

We implement this model in the Rust AgentPProf profiler and evaluate it through three core empirical profiling experiments, RQ1/E1 through RQ3/E3, plus one replayability/scope-control block, RQ4/E4, rather than a chronological run list. RQ1/E1 tests generality and recursive operation-stack formation: across 15 lightly sampled public labeled sources, AgentPProf maps 47,590 samples into the same operation layer, and the same 13,265 operations fold from 9 dataset stacks to 57 phase, 226 semantic, 455 action, or 3,757 fixed-session stacks without changing the input operations. RQ2/E2 is the main hidden-label localization benchmark. On AgentRewardBench, SATraj-OS, AgentNet, and OSWorld-Human, we build six tasks over 34,539 operations and 3,699 positives, score 144 view/ranker policies, and find that task-query operation-stack profiling inspects median 0.0937 top-five work versus 1.0 for flat summaries, improves top-five recall over fixed-session drilldown on 5 of 6 tasks, and reduces median group fragmentation from 285.0 to 157.5 groups. RQ3/E3 isolates mechanisms and profile-configuration actionability. Rank-feature, stack-depth, mapping, transfer, diagnostic-lens, case-packet, and counterfactual analyses show that the useful configuration varies by task and objective: operation-stack views are a Pareto point, not a universal default. Executable profile-spec patches improve AP, top-five lift, and first-positive work on 5 of 6 tasks; a supervised held-out boundary-field ablation improves OSWorld-Human AP from 0.2402 to 0.2583 and reduces groups from 108 to 74, while preserving work counterpoints. RQ4/E4 checks replayability, offline cost, and claim scope. Deterministic replay covers 76 tracked profile specs, and profile-spec input regressions cover local-session, agent-trace, and standard-trace replay. The recorded source provenance, paper-number, rubric, and two-abstraction checks constrain the paper's scope rather than forming a fourth empirical profiling experiment.

The supported claim is therefore a scoped profiling claim: operation-stack profiling can localize, rank, and explain dataset-labeled failures, quality problems, and semantic boundaries in public traces, with less inspection work than flat summaries and a better median-fragmentation tradeoff than the fixed-session drilldown proxy. Fixed-session drilldown is the current trace-tree-shaped baseline; real OpenTelemetry/OpenInference/Phoenix-style span-tree imports remain future baselines. The paper does not claim human-productivity gains, automatic discovery of all intent boundaries, metric dominance, or complete compatibility with existing trace ecosystems.

1 Introduction

AI-agent traces mix many objects that traditional profiling and tracing systems keep separate. A coding agent may ask a model for a plan, call a shell tool, launch a subagent, inspect a file, and retry a failing command. A web or desktop agent may click, type, observe a page, repeat an action, and receive a benchmark label saying that the action was unsafe, redundant, incorrect, or the start of a human subtask. If the profiler fixes the hierarchy at prompt, session, or span boundaries, these questions become hard to ask: which phases concentrate unsafe operations, which action families cause redundant steps, and which groups of operations cross sessions but share the same failure mode?

AgentPProf takes the opposite position. The profiler stores a sequence as operations, where prompts, tool calls, processes, syscalls, GUI actions, and plan steps are operation records or derived fields. Session, span, task, and trace identifiers are fields, exchange containers, or baseline shapes rather than separate profiler objects. Users then choose an operation stack to fold the same operations at different depths. One query may group by `task,phase,action`; another may group by `environment,safety`. Mapping and tagging are first-class field-derivation steps, but they do not add a third profiler object. They only write fields before the stack query runs. Thus the novelty is the abstraction that connects stack shape to aggregation: changing the stack changes what is comparable, localizable, and actionable without changing the underlying trace object model.

This paper makes a profiling claim, not a human-productivity claim. A top-tier profiling paper must show that the profiler attributes, localizes, and ranks problems against explicit workload labels, and that the output leads to optimization insight with better label-scored ranking or inspection cost than baselines. For agents, the claim is:

> *Operation/operation-stack profiling provides label-scored localization and ranking for task-relevant failures, quality problems, and dataset-provided semantic boundaries in real labeled agent traces, while requiring less inspection work than flat summaries and improving the median fragmentation tradeoff relative to fixed-session drilldown.*

We evaluate this claim with public labeled traces rather than a human study. A human or agent analyst study would be useful for productivity claims, but it is not required to test label-scored profiler fidelity. Our evaluation therefore uses hidden labels as the oracle for ranked profile groups and is organized as three empirical profiling experiments plus one replayability/scope-control block.

RQ1/E1 tests operation generality, recursive stack folding, field derivation, and human-boundary alignment under the same two-object model. RQ2/E2 is the main hidden-label localization and ranking benchmark. RQ3/E3 isolates mechanisms and actionability through ranker, stack, mapping, transfer, case, and profile-spec patch ablations. RQ4/E4 checks replayability, offline cost, and claim scope. It is not treated as a fourth hidden-label accuracy result or a fourth empirical profiling experiment. Supplementary results support these blocks rather than defining the paper-facing structure.

2 Design

2.1 Operations

An `operation` is the only sample-level object in AgentPProf. It is a weighted record with string fields. The same representation can encode a prompt, a tool call, an API call, a browser action, a PyAutoGUI action, a process event, or a derived label such as `looping=yes` or `step_correct=incorrect`. External datasets enter the system as operation JSONL. Local Codex or Claude sessions can also be exported as an agent-session trace, imported back, or converted to operation JSONL before profiling.

2.2 Operation Stacks

An `operation stack` is an ordered list of fields. Folding operations by that list yields the usual folded-stack representation, but the stack is chosen at query time. The same operation file can be folded as a flat task summary, a fixed-session tree, a dataset-native hierarchy, a raw action stack, a semantic operation stack, or an oracle label-drilldown. Fixed sessions and spans are therefore baselines and exchange containers, not core profiler abstractions.

2.3 Mapping and Tagging

Mapping and tagging derive new operation fields before folding. For example, desktop actions such as `type` and `key` can map to `phase=input`, while tool/API rows can map to a tool phase before generic action rules run. The Rust CLI exposes inline rules, rule files, profile specs, operation predicates, and stack overrides. `–where` predicates run after mapping and before stack construction, implementing the query predicate without adding a profiler object. Profile specs record operation files, mappings, predicates, views, stacks, and outputs for reproducibility, while command-line flags can still override the stack query.

3 Evaluation Setup

Table 1 summarizes the public trace families. We do not create new test sets, sync complete image corpora, or depend on restricted data. The 15 sources establish generality, while the main hidden-label ranking results use the four oracle-rich families with failure, safety, quality, and human-boundary labels.

For the E2 hidden-label profiler benchmark, we build six tasks: AgentRewardBench looping, AgentRewardBench side-effect, SATraj unsafe operation, AgentNet incorrect step, AgentNet redundant step, and OSWorld-Human group start. Each task has an oracle-positive field, but non-oracle rankers never read that field. We compare six views: flat, fixed-session, dataset-native hierarchy, raw action stack, semantic operation stack, and label-drilldown. We compare four rankers: width, visible-risk, query-aware, and oracle upper bound. Label-drilldown and oracle upper bound are marked as hidden upper bounds.

The metrics match a profiler paper's main claim. Fidelity uses precision@k, recall@operation-budget, F1@k, average precision as a block-level AUPRC-style score, nDCG over positive operation counts, and work-to-first-positive. Fragmentation uses group count, positive group count, and work or groups to reach 50% positive recall. Actionability uses per-task comparisons among stack fields, mapping choices, ranker choices, and oracle headroom. Reproducibility uses repeated Rust `agentpprof –profile-spec` executions over tracked specs and tracked operation JSONL inputs, plus profile-spec input replay regressions for local sessions, exported agent-session traces, and generic Chrome Trace Event imports. The checks cover semantic output hashes, sample counts, unique-stack counts, runtime, output size, effective input metadata, and standard-trace argument import. Build time is excluded from the profiler runtime measurement.

Table 2 gives the paper-facing evaluation structure. The table presents evidence as four reviewer-facing blocks rather than by execution order. Each row names the workload, the headline evidence, and the scoped conclusion supported by that evidence.

The main-paper displays follow this path. Table 1 establishes workload provenance. Table 2 is the four-block claim map. Table 3 checks the two-abstraction and recursive-folding claim. Table 4 and the left side of Figure 1 provide hidden-label fidelity and baseline tradeoff evidence. Table 5 and the right side of Figure 1 provide mechanism/actionability evidence. Tables 6 and 7 provide replay and cost evidence. Supporting materials contain the larger portfolio, case, verdict, and consistency tables; the main text uses them only to substantiate the four evidence blocks.

The rest of the evaluation keeps this structure. E1 establishes that the two profiler objects are sufficient to express heterogeneous traces and recursive folding choices. E2 is the single hidden-label localization/ranking experiment. E3 asks which stack fields, mappings, rankers, and profile specs explain or repair the E2 results. E4 is the replayability, offline-cost, and scope-control block. Additional analyses enter the paper only when they strengthen one of these four blocks as a primary comparison, ablation, counterpoint, provenance check, or scope check. This is the reviewer-facing evidence path for the profiling claim: E1 checks whether the two-object abstraction can represent the workloads and recursive folding choices; E2 is the only primary hidden-label accuracy and baseline tradeoff comparison; E3 explains which mechanisms and profile configurations make the E2 results actionable; and E4 establishes replayability and scope. If the E2 Pareto condition or the E3 mechanism/actionability condition failed, the paper claim would narrow rather than expanding the evidence story.

4 Results

4.1 RQ1/E1: Generality and Recursive Folding

RQ1/E1 tests whether one operation layer can represent heterogeneous agent traces without making prompt, session, tool, process, or syscall records into separate profiler objects. The evaluation

Table 1: Public labeled trace families used by the evaluation. The main E2 hidden-label profiler benchmark uses the four oracle-rich rows in the bottom half.

Family	Native labels	Access path	Evaluation role
WebLINX, Mind2Web, AgentTrek, WebShop	Web actions, demos, tasks, rewards, action history	Hugging Face viewer or public shards	Web navigation coverage
API-Bank, ToolBench, tau-bench	API calls, tool messages, solution paths, outcomes	Hugging Face rows or repo JSONL	Tool/API and dialogue coverage
SWE-agent	Issue trajectory, command actions, target outcome	Hugging Face viewer	Software-agent coverage
AndroidControl, GUI-Odyssey, ScaleCUA	Mobile or GUI actions, instructions, history context	Public/HF samples	Mobile and GUI coverage
AgentRewardBench	Expert success, side-effect, looping, optimality	HF annotations plus cleaned BrowserGym steps	Failure and looping tasks
SATraj-OS	Desktop actions, safety, reward, attack type	HF safety config	Safety task
OSWorld-Human	Human single-action and grouped-action traces	Public GitHub JSON	Human-boundary task
AgentNet	Human desktop PyAutoGUI, step correctness, redundancy	HF repo JSONL streaming	Step-quality tasks

Table 2: Three empirical profiling experiments plus one replayability/scope-control block. Each row gives the workload, headline evidence, and scoped conclusion.

RQ / paper block	Workload	Headline evidence	Scoped conclusion
RQ1/E1: generality and recursive folding	15 public labeled trace families / 47,590 operations, plus local-session and standard-trace exchange fixtures	The same 13,265 operations fold across eight depths, from 9 to 3,757 stacks; leave-dataset-out mapping reduces stacks on 6/9 held-out datasets; 12/12 profile specs remain prompt/session-free; standard-trace exchange round-trips 512 real operations with 512 samples and 11 stacks preserved; boundary backends beat the best simple baseline on 4/5 field-derivation rows.	Two abstractions cover heterogeneous traces; trace containers and mappings are not profiler objects, and field derivation is not automatic intent boundary discovery.
RQ2/E2: hidden-label localization and ranking	Six oracle-backed tasks over AgentRewardBench, SATraj-OS, AgentNet, and OSWorld-Human	The benchmark scores 144 view/ranker policies over 6 tasks, 4 datasets, 34,539 operations, and 3,699 positives. Operation-stack query-aware profiling uses 0.0937 top-five work versus flat 1.0, beats fixed-session top-five recall on 5/6 tasks, and uses 157.5 median groups versus fixed-session 285.	Supported as a hidden-label profiler benchmark with baseline tradeoffs, not human utility.
RQ3/E3: mechanism and actionability	The same six tasks plus held-out OSWorld-Human boundary-backend operations	Executable profile-spec patches improve 5/6 tasks, with median AP delta 0.0376 and top-five lift delta 0.5750. Boundary-derived fields improve OSWorld-Human boundary AP to 0.2583 versus 0.2402 and reduce groups to 74 versus 108, while preserving top-five-work and first-positive-work counterpoints. Rank-feature ablations identify 7 critical and 3 misleading feature rows.	The profiler exposes stack, field, ranker, and profile-spec knobs; it is not an automatic selector or boundary detector.
RQ4/E4: replayability, offline cost, and scope control	76 tracked operation-JSONL profile specs plus local-session, agent-trace, and standard-trace profile-spec input replay regressions	Deterministic replay gives 76/76 semantic and 76/76 raw-byte deterministic specs over 152 invocations, with median runtime 1.601s and p95 2.767s. Profile specs also replay source selection, regex tags, trace inputs, and effective standard-trace args through the same operation-stack path.	The offline profiling artifact is replayable and cheap enough for artifact evaluation; this is systems/reproducibility evidence, not a hidden-label accuracy result, live capture overhead result, human productivity result, or trace-ecosystem compatibility result.

folds the same operation records into task, phase, action, human-boundary, and fixed-session-shaped stacks by changing only mappings and stack fields. Public dataset labels and native trajectory fields provide the oracle signal, with OSWorld-Human boundaries and AgentNet quality labels as the strongest structure labels. The comparison uses dataset-native, no-map, fixed-session, replay, and trace round-trip views. The metrics are coverage, stack range, compression, boundary F1, V-measure, and round-trip equality. The scoped conclusion is configurable operation-stack folding, not complete ecosystem compatibility or latent-intent recovery.

The 14 core public sources produce 42,590 operations. Adding the supplemental ScaleCUA stream brings the smoke set to 47,590 operations. These operations cover web, mobile, desktop, software, API, tool-agent-user dialogue, expert failure labels, safety labels, human grouped-action labels, and desktop step-quality labels. No source required a prompt-specific, GUI-specific, or safety-specific profiler object.

Table 3 is the main E1 display; the following paragraphs unpack the rows rather than introducing additional experiments.

Recursive stack-depth sweep. On the same 13,265 operations, a depth sweep folds to 9 dataset stacks, 57 phase stacks, 226 tool/semantic stacks, 455 action stacks, and 3,757 fixed-session stacks. A fixed session boundary therefore increases stack count by 417.4× relative to dataset depth. On AgentNet, a tracked profile spec folds 16,741 operations into 608 diagnostic stacks, while a CLI stack override changes the output to 83 stacks without changing the operation input. The result supports query-time recursive stack choice rather than a fixed prompt/session hierarchy.

The implementation probes are grouped as interface evidence, not additional main experiments. They show that profile specs can select mapped subsets before folding (729/729, 714/714, and

Table 3: E1 abstraction-evidence summary. The rows use tracked artifacts and show that the same operation layer can be recursively folded, mapped, and boundary-scored without adding prompt/session/tool-specific profiler objects.

Evaluation question	Workload/oracle	Main evidence	Counterpoint / scope
Heterogeneous operation layer	15 public labeled trace families / 47,590 operations	Web, mobile, desktop, software, API, dialogue, failure, safety, human-boundary, and step-quality labels enter as operation fields.	Coverage is lightweight sampling; image-heavy or restricted full benchmarks remain out of scope.
Recursive stack choice	Same 13,265 operations plus AgentNet profile-spec override	9 dataset, 57 phase, 226 tool/semantic, 455 action, and 3,757 fixed-session stacks; AgentNet changes 608 to 83 stacks without changing 16,741 input operations.	Fixed sessions are useful drilldown fields but fragment aggregation when hard-coded.
Field derivation before folding	Held-out mapping and boundary-backend rows	Mapping improves compression 14.049 to 19.091; leave-dataset-out reduces stacks on 6/9 datasets; boundary backends beat the best simple baseline on 4/5 rows.	Mapping coarsens action-to-phase V-measure 0.9343 to 0.7416; looping is explained by `repeat_signal_change`.
Human-boundary folding	OSWorld-Human exact-aligned tasks	4,011 operations / 2,075 human groups; grouped-depth stacks reduce 482 action stacks to 106; group-pattern boundary F1 is 0.627 with precision 1.0.	This is scoped human-group alignment, not automatic recovery of all latent intent boundaries.

4,285/4,285 expected operations), compose operation files, predicates, visible ranking rules, rank modes, and explicit stack depths without prompt/session frames (12/12 specs), and round-trip a real operation-file prefix through a standard trace container while preserving 512 samples and 11 folded stacks. These interface checks are provenance for E1: they test whether the operation layer stays fixed while views change; the localization and ranker gains from those specs are evaluated under E2/E3.

Field derivation and boundary probes. Held-out mapping rules improve compression from 14.049 to 19.091 on 3,990 operations. Leave-dataset-out mapping reduces unique stacks on 6 of 9 held-out datasets with no negative stack-reduction regressions after API/tool phase precedence is fixed. A supervised OSWorld-Human boundary backend reaches F1 0.7735 on held-out adjacent pairs and writes predicted boundary fields that the Rust profiler folds as ordinary operation fields. Boundary backends therefore extend field derivation, not the profiler object model. The field-derivation audit keeps both support and counterpoints visible: held-out mapping improves compression but coarsens action-to-phase V-measure from 0.9343 to 0.7416; profile-spec composition stays prompt/session-free on 12/12 variants; and learned boundary backends beat the best simple baseline on 4/5 rows, while AgentRewardBench looping is fully explained by `repeat_signal_change`. This supports first-class field derivation and suitability checks, not automatic intent boundary discovery and not a new profiler object. Rank-feature criticality from the same audit belongs to RQ3/E3's mechanism/actionability evidence, not to RQ1/E1's abstraction claim.

Human-boundary oracle. OSWorld-Human gives the strongest direct boundary oracle. On 320 exact-aligned tasks with 4,011 operations and 2,075 human groups, grouped-depth stacks reduce 482 action stacks to 106 grouped stacks. A conservative `group_pattern` projection has human-group boundary F1 0.627 with precision 1.0 and recall 0.456. This is not complete intent discovery, but it shows that the same single-action sequence can be folded at action depth or human-group depth and scored against a human boundary oracle.

4.2 RQ2/E2: Hidden-Label Localization and Ranking

RQ2/E2 treats profiling as a ranked localization problem. The benchmark marks 3,699 positive operations among 34,539 operations and asks whether hot stacks and top-ranked groups correspond to those hidden positives. The comparison covers flat summaries, fixed-session drilldown, dataset-native hierarchy, raw-action stacks, operation stacks, label drilldown, and oracle upper bounds. The metrics are precision@k, recall, F1, AP as an AUPRC-style score, nDCG, budget recall and F1, work-to-first-positive, and group count. The scoped claim is a Pareto tradeoff: operation stacks should reduce flat inspection work and improve a useful fixed-session localization or fragmentation objective, while flat and dataset-native views may still win broad-recall or nDCG goals.

Primary comparison and success criterion. E2 treats every profile group as an inspection target and asks whether the ranked targets find hidden positives with less work than flat summaries and less fragmentation than a fixed-session trace-tree proxy. Success is therefore a Pareto condition, not a single winning metric: operation stacks must reduce flat inspection work, improve a meaningful fixed-session localization or fragmentation objective, and keep fixed-session early-hit counterpoints visible. Table 4 is the main E2 localization display; supporting case views explain the result without becoming separate paper experiments. Flat summaries recover all positives, but only by inspecting the whole task, so their median work@5 and work-to-first-positive are both 1.0. Fixed-session drilldown finds an early positive with median work-to-first-positive 0.0044 and median work@5 0.0163, but its median recall@5 is only 0.0233 and it fragments the workload into 285.0 groups. Query-aware operation stacks inspect median 0.0937 work in the top five groups, reach median recall@budget30 of 0.3900, and reduce the median group count to 157.5.

Failure interpretation and baseline scope are part of the result. Fixed-session drilldown is the evaluated trace-tree-shaped baseline, not evidence of full OpenTelemetry/Phoenix/LangSmith/Perfetto compatibility. On the same scored rows, operation-stack query-aware improves fixed-session top-five recall and F1 on 5/6 tasks, budget-30 recall on 4/6, and group count on 4/6, while fixed-session retains lower top-five work and first-positive work on 4/6. The depth-aware slice gives the finer version of the same claim: operation-stack rankings beat fixed-session on budget-30 positive-unit recall for 20/24 task-depth rows and use fewer groups to reach 50% positive units on 22/24 rows. If these counterpoints disappeared from the paper, the result would be overstated; with them, E2 supports a better localization-budget/fragmentation tradeoff, not dominance over every trace-tree-shaped objective.

The same scored outputs are also shown through non-flamegraph paper views instead of making flamegraphs the only presentation.

Table 4: E2 hidden-label localization benchmark. Hidden policies read oracle labels and are included only as upper bounds. WTFP is work-to-first-positive.

Policy	Hidden	AP	nDCG	P@5	R@5	F1@5	Work@5	R@30%	WTFP
flat:width	0	0.1678	1.0000	0.1678	1.0000	0.2868	1.0000	0.0000	1.0000
fixed-session:query-aware	0	0.3476	0.7642	0.2555	0.0233	0.0427	0.0163	0.3559	0.0044
dataset-native:query-aware	0	0.3572	0.7445	0.1712	0.8665	0.2822	0.8539	0.3377	0.0582
raw-action:query-aware	0	0.2744	0.5012	0.2513	0.2442	0.2195	0.1507	0.3325	0.0374
operation-stack:width	0	0.1610	0.9364	0.1398	0.4746	0.2147	0.5424	0.3372	0.1694
operation-stack:query-aware	0	0.3117	0.5487	0.1991	0.1880	0.1981	0.0937	0.3900	0.0378
operation-stack:oracle-upper	1	0.5993	0.6372	0.8984	0.3004	0.4029	0.0441	0.5640	0.0122
label-drilldown:oracle-upper	1	1.0000	1.0000	1.0000	0.6703	0.7972	0.1150	1.0000	0.0377

The main body keeps one representative figure because it directly serves E2 and E3: the left panel shows the localization/work/fragmentation tradeoff, and the right panel shows which profile-configuration knobs the evidence activates. The remaining portfolio table, headline rows, task case cards, and verdict matrix stay in the supporting views as explanatory material rather than additional main-body experiments.

The supporting case and verdict views still matter, but the main text uses them as checks on the three core experiments rather than as separate tables. They report that all six labeled tasks improve AP and top-five inspected work over flat summaries, five improve top-five recall over fixed-session drilldown, four reduce fixed-session group fragmentation, and all six have an actionable configuration result through either an accepted profile-spec patch or the boundary-field repair. They also keep the negative evidence visible: fixed-session remains the first-positive or group-count counterpoint on several tasks, and OSWorld-Human requires boundary-derived fields rather than visible phase/action ranking alone.

The E2 result is deliberately a Pareto claim, not universal dominance. Operation stacks improve top-five recall over fixed-session query-aware drilldown on 5 of 6 tasks and reduce median group fragmentation from 285.0 to 157.5 groups, but fixed-session often finds the first positive with less work. Operation stacks also save work against flat summaries on all tasks, while flat remains the full-recall counterpoint and dataset-native hierarchy can reach high recall at high inspection work. The supporting bootstrap, label-permutation, work-budget, fragmentation, and depth-aware audits keep this tradeoff honest rather than adding separate experiments: 10,000 paired bootstrap resamples show stable AP, work, budget-recall, and group-count deltas; 2,000 label-permutation trials show AP exceeds the 95% prevalence/group-size null on 6/6 tasks; the 30% work-budget curve reaches median recall 0.3900 versus 0.3559 for fixed-session and 0.0000 for flat; and the fragmentation audit uses fewer groups on 5/6 tasks at the same budget while preserving fixed-session first-positive-work counterpoints. Depth-aware scoring then repeats the comparison at dataset-provided oracle depths: operation-stack query-aware improves fixed-session budget-30 positive-unit recall on 20/24 task-depth rows and uses fewer groups to reach 50% positive units on 22/24. Together, these slices support a faithful localization surface with better inspection-work and fragmentation tradeoffs than flat or fixed-session alone; they do not support a single best hierarchy, a human-productivity claim, or dominance over every trace-tree-shaped objective.

4.3 RQ3/E3: Mechanism and Actionability

RQ3/E3 asks why the E2 rankings improve and whether the profiler points to concrete configuration changes. The candidate mechanisms are stack fields, mapping and tagging rules, rankers, profile specs, and boundary-derived operation fields. Hidden labels score the output only after profiling, and held-out OSWorld-Human boundary positives score boundary-derived fields. The comparison covers default and patched specs, visible rankers, transfer policies, learned-boundary fields, fixed-session drilldown, and semantic-width stacks. The metrics are accepted patches, AP delta, top-five lift, first-positive work, group reduction, and top-five-work counterpoints. The scoped conclusion is profile-configuration actionability, not automatic boundary discovery, automatic patch selection, or a universal selector.

Primary mechanism question. E3 starts from the E2 ranked outputs and asks whether the profiler can explain what to change in the profile configuration. Hidden labels are used only after profiling to score a configuration; the visible inputs are stack fields, mapping/tagging rules, rank features, profile specs, and boundary-derived operation fields. The success criterion is an actionable, replayable diagnosis: a task should point to a concrete stack, field, ranker, mapping, profile-spec, or drilldown change, and executable patches should improve the scored output without turning into an automatic selector.

Table 5 is the main E3 display. It compresses the task-card evidence into a paper-facing diagnosis table; the supporting case views remain provenance rather than additional main experiments. SATraj unsafe is the cleanest operation-stack win: environment, phase, and action fields plus query-aware risk ranking give the best visible top-five F1. Looping exposes the value and limit of `repeat_signal`: it helps ranking, but positives are prevalent, so prevalence-aware ranking remains the optimization. Side-effect has large oracle headroom and points to deeper write/input mappings. AgentNet step-quality tasks need action-family localization followed by fixed-session examples. OSWorld-Human needs boundary-derived fields rather than action depth alone. Thus the first actionability result is not that one view wins; it is that the profiler identifies which stack field, mapping, ranker, or drilldown should change for each task family. The task-level verdict synthesis clarifies the claim boundary: all six tasks support E3 with counterpoints, not as unconditional wins. Five of six tasks have accepted executable profile-spec patches; the remaining OSWorld-Human row points to boundary-derived fields and keeps top-five and first-positive work counterpoints. The six rows resolve into three reusable optimization patterns. Safety and side-effect tasks point to field and ranker

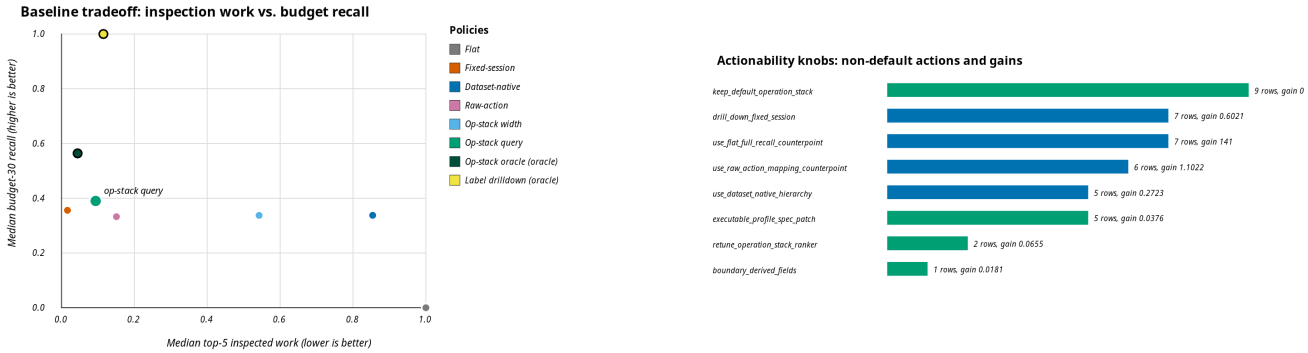

Figure 1: Two non-flamegraph paper views. Left: the RQ2/E2 baseline tradeoff places operation-stack query-aware profiling in the low-inspection-work region while preserving fixed-session and oracle counterpoints. Right: the RQ3/E3 actionability view turns the same scored outputs into profile-configuration knobs, showing that many objectives require changing the view, ranker, profile spec, or boundary-derived fields rather than using one fixed default view.

changes because environment, phase/action, and write/input fields move relevant positives into smaller inspected regions. Step-quality tasks point to a drilldown workflow because desktop phase, repeat, and action fields locate suspicious regions before fixed-session examples explain individual traces. Human-boundary tasks point to representation repair because action-depth stacks are insufficient, while boundary-derived operation fields improve AP and reduce groups with explicit inspection-work counterpoints. These patterns are configuration guidance over profile inputs, stack fields, and rankers, not labels predicted by the profiler.

The Rust mechanism ablations isolate where those diagnoses come from without turning the paper into a run list. First, stack-text rules and rank-mode variants are replayable from profile specs but too coarse for safety and side-effect tasks. Second, operation-level rank features close that gap by matching visible mapped operation tokens before folding: semantic-stack feature ranking improves AP over width on 5 of 6 tasks, top-five lift on 4 of 6, and first-positive work on 5 of 6; coarse depth improves AP on 4 of 6 while reducing groups. Third, leave-one-feature and robustness ablations make the knobs concrete: `repeat_signal`, write-action features, and `status=success` are critical in different families, while misleading safety-like or input-phase features can hurt AP or first-positive work. Equal/global feature banks often preserve the gains, and guided repairs improve AP on 2 of 3 misleading-feature cases, but these repairs use offline scored feedback and are not an automatic learned ranker. These audits are provenance for this single E3 mechanism block.

The view and policy audits show the same mechanism at the analysis-surface level. Best AP and best top-five F1 are split across operation-stack, fixed-session, dataset-native, raw-action, and flat views; leave-task source selection cannot reliably pick the best visible view. Nevertheless, the operation-stack query-aware policy remains Pareto-frontier on all six tasks, beats flat top-five work on all six, beats fixed-session top-five recall on 5 of 6, and reaches fewer groups-to-50% positives than fixed-session on 5 of 6. At a 25% recall target it covers all six tasks with median work 0.2000 versus flat 1.0000 and median 16.0 groups versus fixed-session 50.0; at 50% recall, other depths or views become the best-work counterpoints. Sequence and oracle-depth audits add the inspection unit below and above individual groups. Sequence and oracle-depth scoring show that at a 30% operation budget, operation-stack query-aware reaches median 0.4669 positive-session recall versus fixed-session 0.3230, and across 24 oracle-depth rows it has median 0.4342 budget-30 positive-unit recall, beats fixed-session unit recall on 20/24 rows, and reaches 50% positive units with fewer groups on 22/24. Positive-run proxy units are derived episode units, not human intent boundaries. The fixed-session depth-gap counterpoint remains, so this is depth-aware triage, not complete intent-boundary recovery. The multi-metric audit also keeps the metric surface honest: it records 30 supporting baseline-metric comparisons, 16 explicit counterpoints, and nDCG cases where flat or dataset-native views can win.

The actionability portfolio turns those comparisons into reviewer-auditable profile-configuration tuning insight. Across 36 objective rows, every row has a concrete configuration action, but 27/36 best visible policies are not the default operation-stack query-aware view; transfer misses are mostly view or ranker changes rather than opaque errors. The same evidence is presented as diagnostic lenses, case packets, and baseline contrasts: six lenses cover 36 objectives, top-five case groups contain positives on 6/6 tasks, top-one groups on 5/6, median top-five lift is 1.6508 at 0.0937 work, and operation stacks beat flat top-five work on 6/6 and fixed-session top-five recall on 5/6 while preserving fixed-session first-positive and flat full-recall counterpoints. Counterfactual and held-out transfer checks separate visible actions from target-label leakage: 36/36 best rows are visible non-oracle, 27/36 require a non-default action, 25/36 require a view change, held-out action transfer stays within tolerance on 35/60 aligned decisions, but exact action-class transfer is only 7/60 overall and 2/42 on non-default target actions. These support audits form one E3 actionability block. The actionable claim is therefore a tunable diagnosis surface, not a label-free automatic selector.

Finally, executable patches close one loop from diagnosis to profiler configuration. Default and profile-guided Rust profile specs run over the same visible operation input, and hidden labels are

scored only after `agentpprof -profile-spec`. Five of six patches improve AP, top-five lift, and first-positive work, with median AP delta 0.0376 and median top-five lift delta 0.5750. The failure interpretation matters here too: the rejected OSWorld-Human patch shows that visible phase/action rank features are insufficient for a human-boundary objective. Folding held-out boundary-derived operation fields then improves AP to 0.2583 versus 0.2402 for semantic width and reduces groups to 74 versus 108, while top-five work and first-positive work increase. These patch and boundary-field results support boundary fields as an actionable operation-field knob, not automatic boundary discovery or automatic patch selection. The result belongs to mechanism/actionability, not a fifth core experiment.

4.4 RQ4/E4: Reproducibility, Offline Cost, and Scope Control

RQ4/E4 asks a narrower question than E2/E3: can reviewers replay the offline profile-spec path at low local cost while checking that source provenance, paper numbers, two-abstraction wording, and non-claims remain aligned? The evidence is the tracked specs and inputs, profile-spec input replay regressions, repeated profile outputs, runtime logs, and recorded source provenance. We compare default output with deterministic-output replay, semantic profile hashes with raw-byte hashes, and effective input metadata with silently ignored source paths. The measured quantities are deterministic spec pass rate, profiler invocations, median and p95 runtime, sample equality, stack equality, raw-byte output equality, and input-source replay coverage. These checks bound the paper claim. They are not a hidden-label accuracy result, live eBPF overhead, human utility, complete ecosystem compatibility, or a universal selector.

The empirical content of this subsection is only one replayability/cost block. It reruns the profile specs used by the labeled-trace experiments: 76 tracked specs over tracked operation JSONL inputs, executed twice for 152 profiler invocations. The semantic profile content is stable for 76/76 specs in both modes; deterministic output also makes raw-byte profile files stable for 76/76 specs by fixing profile timestamps. The clean deterministic rerun reports median runtime 1.601s and p95 2.767s per spec. The replayed files provide provenance for this E4 block. This supports replayable offline profiler outputs, not live eBPF overhead or analyst productivity, or another empirical profiling experiment.

Input-source replay checks belong to the same E4 block. The Rust profile-spec path carries `session_files`, `trace_files`, `standard_trace_files`, regex tag rules, predicates, rank rules, stack overrides, and `include_standard_stacks`. The regressions replay the public Codex fixture as a local session and as an exported agent-session trace, then replay a generic Chrome Trace Event import whose non-AgentSight `protocol` argument becomes an operation field before folding. These checks keep local sessions and trace containers inside the same operation / operation-stack path. They do not add a new accuracy experiment, a third profiler object, or an ecosystem-compatibility claim.

Tables 6 and 7 are the two E4 main displays: they summarize the replayability/cost result and the deterministic-output repair, while their row labels name source-spec families rather than additional accuracy experiments.

The remaining consistency checks support reproducibility and scope control, not new empirical evidence. They reuse tracked inputs, tracked outputs, and the current paper text without reranking, downloading, syncing, creating, relabeling, rerunning the profiler, or running a human or agent analyst task. The checks verify replay paths, source provenance, paper-number alignment, two-abstraction wording, and non-claim consistency. They let reviewers audit whether the paper is faithful to tracked outputs, but they do not increase the localization, actionability, or overhead claims.

Profile-spec determinism and local cost. The replay workload reruns the existing profile specs used by the E2 and E3 views, rankers, and robustness checks. The workload contains 76 tracked specs: four view specs, 12 operation-feature rank specs, and 60 robustness specs. Each spec is executed twice by the Rust profiler against tracked operation JSONL inputs, for 152 profiler invocations. No dataset is downloaded or synced. The replay check hashes the semantic profile content after removing the JSON `generated_at` timestamp, and separately reports raw-byte hashes.

All 76 specs reproduce the same semantic profile hash, sample count, and unique-stack count across repetitions. Raw-byte determinism is only 4 of 76, because JSON outputs intentionally include a generation timestamp; the stable semantic hash keeps the reproducibility claim on the profile content. Median output size is 37,546 bytes, the largest output is 306,099 bytes, and the largest profile has 2,012 unique stacks. This supports the offline profiler path and profile-spec reproducibility claim, not live eBPF overhead or human utility.

The deterministic-output replay closes the raw-byte caveat with an explicit deterministic-output mode. The Rust CLI accepts `-deterministic-outp` and profile specs may set `deterministic_output`; the mode replaces JSON `generated_at` and pprof profile time with fixed values while leaving profile content unchanged. It reruns the same 76 specs and 152 invocations with this flag. Both semantic and raw-byte determinism are 76 of 76. The clean rerun records empty git and code status. The median runtime in this run is 1,601.0827 ms and the p95 is 2,767.1653 ms; we report these as offline replay costs for this run, not as a performance comparison against the default-output replay.

5 Related Work and Novelty

Classic profiling and trace analysis. Flame graphs and pprof establish the folded-stack lineage for profiling: profiles consist of weighted samples attached to a location hierarchy, can carry sample labels, and pprof can visualize labels as pseudo stack frames through tag-root or tag-leaf options [1, 2]. Perfetto generalizes trace ingestion and SQL analysis over many trace formats and can derive new events from trace contents [3]. These systems are closer than a simple "fixed hierarchy" baseline. AgentPProf therefore does not claim novelty for query-time aggregation alone. The difference is the domain object and evaluation target: the sample is an agent operation, the frames are recursive multi-field operation projections, and the groups are scored against hidden agent-trajectory labels across datasets.

Agent tracing and LLM observability. OpenTelemetry GenAI and OpenInference standardize GenAI spans, LLM calls, agent reasoning steps, tool invocations, retrieval operations, MCP, and provider-specific telemetry [4, 5]. LangSmith, Langfuse, and Phoenix

Table 5: E3 diagnosis/actionability summary. Each row connects a label-scored localization signal, a concrete profile-configuration change, and the scoped verdict/counterpoint. This table compresses task-card and verdict evidence; it is not a new empirical experiment.

Task	Localization signal	Profile action	Verdict / counterpoint
Looping	Work@5 0.4938, R@5 0.6508, patch AP +0.1155	Keep `repeat_signal` in the stack, but add prevalence-aware ranking because positives are common.	Supports, with counterpoint: raw-action finds the first positive with less work.
Side-effect	Work@5 0.1454, R@5 0.1139, patch AP +0.0591	Increase write/input weights or use a deeper side-effect mapping before ranking.	Supports, with counterpoint: fixed-session has fewer groups; raw action wins top-five lift.
Unsafe operation	Work@5 0.0420, R@5 0.2621, patch AP +0.5081	Use environment, phase, and action stack fields; prioritize risky environments and write actions.	Supports, with counterpoint: fixed-session finds the first positive with less work.
Incorrect step	Work@5 0.0014, R@5 0.0034, patch AP +0.0160	Use desktop environment, phase, repeat, and action fields, then drill into fixed sessions for examples.	Supports, with counterpoint: fixed-session remains a drilldown baseline.
Redundant step	Work@5 0.0089, R@5 0.0177, patch AP +0.0011	Use desktop environment, phase, repeat, and action fields, then drill into fixed sessions for examples.	Supports, with counterpoint: fixed-session finds the first positive with less work.
Group start	Work@5 0.4074, R@5 0.3874, boundary AP 0.2583	Use group-depth or boundary-derived fields; action-depth alone fragments starts.	Supports, with counterpoint: boundary fields reduce groups to 74 but increase top-five and first-positive work.

Table 6: E4 replayability/cost main display. Runtime is seconds per spec, measured after the Rust binary exists. Semantic hashes exclude the JSON `generated_at` field; raw-byte hashes are reported separately.

Workload	Specs	Med. s	P95 s	Med. stacks
View specs	4	3.448	4.273	631.0
Rank-feature specs	12	1.539	2.692	41.0
Robustness specs	60	1.542	2.640	41.0
All specs	76	1.581	2.719	42.0

Table 7: E4 deterministic-output main display comparing default and `--deterministic-output` replay over the same 76 profile specs.

Config	Mode	Sem.	Raw	Med. s	P95 s
Default	default	76/76	4/76	1.581	2.719
Deterministic	deterministic	76/76	76/76	1.601	2.767

expose traces, sessions, observations, evaluations, dashboards, datasets, and prompt iteration workflows [6–8]. AgentOps gives the closest academic same-problem framing by arguing that agent observability should trace goals, plans, tools, artifacts, and associated data [9]. These systems are strong prior work, so the safe novelty claim is not generic "agent observability." In this paper, traces, spans, sessions, and observations are exchange containers or baseline shapes. E2 evaluates fixed-session drilldown as the current trace-tree-shaped baseline; real OpenTelemetry/OpenInference or Phoenix span-tree imports remain future interoperability baselines. The profiler abstractions are only operations and operation stacks, and the central mechanism is that stack shape determines semantic aggregation at query time.

Agent failure and span localization. Recent agent-diagnosis work is the closest same-problem threat. AgentRx releases failed trajectories with critical-failure-step and failure-category annotations and uses constraint checking plus an LLM judge to localize root causes [22]. TELBench/DRIFT builds a deep-research span-localization benchmark from semantic spans annotated for harmful errors [23]. Holistic Evaluation and Failure Diagnosis pairs agent-level diagnosis with per-span evaluation over long traces [24]. AgentAtlas argues that outcome leaderboards miss trajectory and control-decision quality [25]. These works make clear that failure localization and trajectory diagnosis are not novel by themselves. The narrower difference here is that AgentPProf treats profile groups as profiler outputs: the same operations are folded into flat, fixed-session drilldown, dataset-native, raw-action, and operation-stack views, then scored with AP, nDCG, recall under inspection budgets, work-to-first-positive, fragmentation, and profile-configuration actionability across heterogeneous public labeled traces. AgentRx- or TELBench-style traces are therefore future oracle/baseline candidates, not current evidence for a broader span-localization claim.

Public labeled agent trajectories. AgentRewardBench, OSWorld-Human, OpenCUA/AgentNet, SATraj-OS, and OSWorld provide the labels and workloads that make the hidden-label profiler benchmark possible [12, 18–21]. Prior benchmark papers use these labels to evaluate agents, reward models, safety behavior, or efficiency. We use them as hidden oracles for a profiler benchmark: different stack-shaped aggregates become ranked outputs, and the evaluation measures precision, recall, AP, nDCG, inspection work, fragmentation, and profile-configuration tuning insight. This is the paper's scoped novelty.

6 Discussion

The result scope is deliberately narrow. The main claim is a profiler claim: on six real labeled tasks, operation-stack profiles can be scored as ranked localization outputs against dataset-provided labels, and they expose a work/fragmentation/actionability trade-off against flat summaries, fixed-session drilldown, dataset-native hierarchy, and raw-action stacks. The supporting audits add uncertainty checks, label-permutation negative controls, view/depth task-fit checks, fixed-recall target costs, sequence-scope scoring, oracle-depth scoring, action counterfactuals, transfer checks, and replayability checks. They do not add a new empirical claim; they explain when the headline comparison should be trusted and when it should be narrowed.

The strongest negative results define the useful boundary. Operation-stack query-aware ranking improves AP over width on all six E2 tasks, but top-five F1, first-positive work, and high-recall targets remain task dependent. Mapping helps some tasks and hurts others; feature ablations find both critical and misleading fields; held-out action transfer is often within tolerance but rarely transfers the

exact action class for non-default targets. Fixed-session drilldown remains a real first-positive counterpoint, raw-action stacks can cover broader session scope, and OSWorld-Human needs boundary-derived fields. These counterpoints are why the design exposes stack fields, mapping rules, rank modes, operation-level rank features, profile specs, and task-specific policies instead of hard-coding one hierarchy.

These results do not show that humans or agents answer questions faster with the tool, nor do they measure live eBPF overhead. They also do not show automatic discovery of all intent boundaries, a label-free automatic selector, or complete compatibility with OpenTelemetry, Phoenix, LangSmith, Langfuse, or Perfetto producers. Those systems remain related work and baselines. In this paper, trace formats are containers; fixed sessions are the evaluated drilldown baseline, while real span-tree imports remain future baselines. The profiler abstractions remain only operations and operation stacks.

7 Conclusion

AgentPProf reduces agent profiling to two abstractions. Operations carry all observed and derived fields; operation stacks fold those fields at the depth chosen by the query. Mapping, predicates, rank rules, profile specs, trace exchange, and boundary backends are mechanisms around those two objects, not additional profiler abstractions.

Across 15 public labeled trace sources, the model covers web, mobile, desktop, software, API, dialogue, failure, safety, human-boundary, and step-quality trajectories. On the four oracle-rich families, operation-stack profiling localizes and ranks labeled problems with less inspection work than flat summaries and a better median-fragmentation tradeoff than fixed-session drilldown, while the mechanism audits show which fields, mappings, rankers, depths, and profile-spec patches produce the gains. The paper's central claim is therefore a profiler claim: operation/operation-stack profiling exposes dataset-labeled task-relevant failures, quality problems, and semantic boundaries, and produces profile-configuration actionability without relying on prompt/session/span boundaries. The novelty is not a new renderer, trace schema, or generic query engine; it is the combination of agent-operation records, recursive multi-field stack projections, and hidden-label profile-group scoring.

The evidence remains scoped. The profile-spec and deterministic-output audits make the offline outputs reproducible, and the claim-scope checks keep the paper aligned with tracked results. They do not claim human or agent analyst productivity, live eBPF overhead, automatic discovery of all intent boundaries, a label-free universal selector, or complete compatibility with OpenTelemetry, Phoenix, LangSmith, Langfuse, or Perfetto producers.

References

[1] Brendan Gregg. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.
[2] Google pprof documentation. https://github.com/google/pprof/blob/main/doc/README.md.
[3] Perfetto Trace Processor documentation. https://perfetto.dev/docs/analysis/trace-processor.
[4] OpenTelemetry GenAI semantic conventions. https://github.com/open-telemetry/semantic-conventions-genai.
[5] OpenInference specification. https://arize-ai.github.io/openinference/spec/.
[6] LangSmith Observability. https://docs.langchain.com/langsmith/observability.
[7] Langfuse Observability. https://langfuse.com/docs/observability/overview.
[8] Arize Phoenix documentation. https://arize.com/docs/phoenix.
[9] Liming Dong, Qinghua Lu, and Liming Zhu. AgentOps: Enabling Observability of LLM Agents. https://arxiv.org/abs/2411.05285.
[10] McGill-NLP WebLINX. https://huggingface.co/datasets/McGill-NLP/WebLINX.
[11] OpenGVLab GUI-Odyssey. https://huggingface.co/datasets/OpenGVLab/GUI-Odyssey.
[12] WukLab OSWorld-Human. https://github.com/WukLab/osworld-human.
[13] xlangai AgentNet. https://huggingface.co/datasets/xlangai/AgentNet.
[14] McGill-NLP AgentRewardBench. https://huggingface.co/datasets/McGill-NLP/agent-reward-bench.
[15] AI45Research SATraj-OS. https://huggingface.co/datasets/AI45Research/SATraj-OS.
[16] AgentSuite tau-bench trajectories. https://huggingface.co/datasets/AgentSuite/tau-bench-trajectories.
[17] OpenGVLab ScaleCUA-Data. https://huggingface.co/datasets/OpenGVLab/ScaleCUA-Data.
[18] Xing Han Lù et al. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. https://arxiv.org/abs/2504.08942.
[19] Tianbao Xie et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. https://arxiv.org/abs/2404.07972.
[20] Xinyuan Wang et al. OpenCUA: Open Foundations for Computer-Use Agents. https://arxiv.org/abs/2508.09123.
[21] Shanghai AI Lab. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. https://arxiv.org/abs/2605.06230.
[22] Shraddha Barke et al. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. https://arxiv.org/abs/2602.02475.
[23] NJU-LINK. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. https://arxiv.org/abs/2606.02060.
[24] Holistic Evaluation and Failure Diagnosis of AI Agents. https://arxiv.org/abs/2605.14865.
[25] Parsa Mazaheri and Kasra Mazaheri. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. https://arxiv.org/abs/2605.20530.